



University of Information Technology and Sciences (UITS)

Department of Computer Science and Engineering

COURSE: Software Project Design and Development (CSE 416)

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: CampusCare

Centralized University Complaint Management System

Submitted To:

Al-Imtiaz

Associate Professor & Head
Ph.D. (Research Fellow), BUET
Department of CSE, UITS

**Submitted By (Group
Members):**

Rudro Antony Mrong

ID: 0432320005101059

Md. Masud Rahman

ID: 0432320005101064

Submission Date: August 2026

Abstract

CampusCare is an internal, web-based 3-tier ticketing and facility helpdesk application designed specifically for university campus infrastructure management. In many higher education institutions, maintenance reporting relies on unorganized communication channels such as verbal walk-ins, phone calls, and unmonitored emails. This fragmentation leads to misplaced requests, delayed issue resolutions, zero progress visibility for reporters, and a complete lack of administrative oversight regarding institutional asset health.

CampusCare addresses these operational challenges by implementing a centralized digital pipeline governed by a strict state machine workflow (**Submitted** → **Assigned** → **In Progress** → **Resolved** → **Closed / Reopened**). Built using modern full-stack web technologies—including Next.js (App Router, React 19, TypeScript), Tailwind CSS, Shadcn UI, and Supabase (PostgreSQL engine with Row Level Security policies and Supabase Storage)—the platform provides isolated, role-based interfaces for Students (End-Users), Staff (Resolvers), and Administrators (Dispatchers & Analysts).

This Software Requirements Specification (SRS) document presents a comprehensive analysis of CampusCare, detailing the functional requirements, non-functional performance benchmarks, relational database entity schemas, behavioral system models (Use Case, Class, State Machine, ER, Sequence, and Data Flow Diagrams), and a traceably mapped system testing plan.

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Document Conventions	1
1.3	Definitions, Acronyms, and Abbreviations	1
1.4	Intended Audience and Reading Suggestions	2
1.5	Product Scope	2
1.6	References	3
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	3
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
2.6	User Documentation	5
2.7	Assumptions and Dependencies	5
3	External Interface Requirements	5
3.1	User Interfaces	5
3.2	Hardware Interfaces	5
3.3	Software Interfaces	5
3.4	Communications Interfaces	6
4	System Features (Functional Subsystems)	6
4.1	Student Module: Complaint Submission & Tracking Engine	6
4.1.1	Description and Priority	6
4.1.2	Stimulus / Response Sequences	6
4.1.3	Functional Requirements	6
4.2	Staff Module: Task Queue & SLA Resolution Workflow	7
4.2.1	Description and Priority	7
4.2.2	Stimulus / Response Sequences	7
4.2.3	Functional Requirements	7
4.3	Admin Module: Dispatcher Command Center & Audit Ledger	7
4.3.1	Description and Priority	7
4.3.2	Stimulus / Response Sequences	8
4.3.3	Functional Requirements	8
4.4	Resolution Verification & Student Feedback Subsystem	8
4.4.1	Description and Priority	8
4.4.2	Stimulus / Response Sequences	8
4.4.3	Functional Requirements	9
5	Other Nonfunctional Requirements	9
5.1	Performance Requirements	9
5.2	Safety Requirements	9
5.3	Security Requirements	9
5.4	Software Quality Attributes	10
5.5	Business & Operational Rules	10

5.6	Resource & Cost Estimation	10
6	System Design & Behavioral Diagrams	10
6.1	Use Case Diagram	11
6.2	Class Diagram	12
6.3	Ticket State Machine Transition Diagram	13
6.4	Entity-Relationship (ER) Diagram	14
6.5	Sequence Diagram	15
6.6	Data Flow Diagrams (DFD Level 0 & Level 1)	16
7	Database Design & Relational Schemas	17
7.1	PostgreSQL Relational Table Definitions	17
7.1.1	Table 1: <code>users</code> (Profiles)	17
7.1.2	Table 2: <code>departments</code>	17
7.1.3	Table 3: <code>categories</code>	17
7.1.4	Table 4: <code>complaints</code>	17
7.1.5	Table 5: <code>progress_notes</code>	18
7.1.6	Table 6: <code>attachments</code>	18
7.1.7	Table 7: <code>feedback</code>	19
7.1.8	Table 8: <code>audit_logs</code> (Immutable Ledger)	19
7.2	Database Triggers & Row Level Security (RLS) Access Policies	19
7.2.1	Complaint Number Generator Trigger	19
7.2.2	Automated Audit Log Trigger (With System Task Fallback)	20
7.2.3	Row Level Security (RLS) Sample Policy	20
8	System Testing & Quality Assurance	21
8.1	System Test Plan & Mapping	21
8.2	Requirements Traceability Matrix	21
8.3	Detailed Test Cases	22
8.3.1	Test Case TC-01: Student Authentication and Session Protection	22
8.3.2	Test Case TC-02: File Attachment Type and Size Enforcement	22
8.3.3	Test Case TC-03: Unique Complaint ID Trigger Verification	22
8.3.4	Test Case TC-04: Student RLS Isolation Verification	22
8.3.5	Test Case TC-05: Staff Department Queue Isolation	23
8.3.6	Test Case TC-06: Ticket Acceptance State Transition	23
8.3.7	Test Case TC-07: Repair Proof Upload and Resolution Marking	23
8.3.8	Test Case TC-08: Admin Staff Reassignment	23
8.3.9	Test Case TC-09: PostgreSQL Automated Audit Log Insertion	24
8.3.10	Test Case TC-10: Student Resolution Confirmation	24
8.3.11	Test Case TC-11: Dashboard Core Web Vitals Benchmark	24
8.3.12	Test Case TC-12: SQL Injection & XSS Input Sanitization	24
9	User Interface Wireframe Specifications	25
9.1	Student Portal Layout Specification	25
9.2	Staff Resolver Workspace Layout Specification	25
9.3	Admin Command Center Layout Specification	25

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the detailed functional, non-functional, interface, database, and behavioral requirements for **CampusCare: Centralized University Complaint Management System**. This document follows the IEEE 830 standard guidelines and acts as an authoritative baseline for software engineers, database architects, UI/UX designers, quality assurance testers, and project instructors.

1.2 Document Conventions

This document observes standard typography and formatting conventions:

- **Bold Text:** Indicates UI labels, role designations, entity names, or critical emphasis.
- **Monospace Text:** Denotes database table names, SQL attributes, route paths, environment parameters, or state machine statuses.
- **Requirement Identifiers:** All functional and non-functional requirements observe structured, readable numbering schemes (e.g., **Requirement 4.1.A**, **Requirement 5.3.B**) to facilitate backward and forward traceability in testing.

1.3 Definitions, Acronyms, and Abbreviations

To facilitate clarity across technical and evaluation audiences, the acronyms and terms used throughout this document are defined in Table 1.

Table 1: Centralized Glossary of Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification (IEEE 830 standard document).
RBAC	Role-Based Access Control (Restricting access based on user role).
SLA	Service Level Agreement (Target resolution timeframe based on urgency).
RLS	Row Level Security (PostgreSQL engine-level database access control).
RSC	React Server Components (Next.js server-rendered component paradigm).
UUID	Universally Unique Identifier (128-bit database primary key standard).
JSONB	Binary JSON storage format in PostgreSQL supporting indexed sub-queries.
KPI	Key Performance Indicator (Aggregate quantitative performance metric).
DFD	Data Flow Diagram (Process and data store architecture model).
ERD	Entity-Relationship Diagram (Database schema relationship model).
UITS	University of Information Technology and Sciences.
BUET	Bangladesh University of Engineering and Technology.

1.4 Intended Audience and Reading Suggestions

This document is designed for the following stakeholders:

- **Course Instructor & Evaluators:** To verify project scope, engineering rigor, requirement completeness, and design correctness for CSE 416.
- **Frontend & Backend Developers:** To guide UI component implementation, Next.js Server Actions, and Supabase database interactions.
- **Quality Assurance Testers:** To execute test plans against the traceability matrix in Section 8.

1.5 Product Scope

CampusCare is a centralized university facility helpdesk system designed to replace manual, paper-based, or fragmented communication channels. The product scope includes:

1. **Student Module:** Authenticated complaint submission with optional image upload, human-readable ID generation (CMP-YYYY-XXXX), real-time status timeline tracking, and resolution confirmation/rejection loop with optional anonymous star ratings.
2. **Staff Module:** Departmental task queue, SLA deadline indicators, status transition controls, repair completion proof upload, and progress note feeds.

3. **Admin Module:** Global command center with multi-parameter ticket search/filtering, manual ticket assignment, priority escalation, user management, category/department taxonomies, analytics charts, and immutable audit logs.
4. **Data Core:** Relational PostgreSQL database with Row Level Security (RLS), foreign key integrity, and trigger-driven audit logging.

1.6 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
2. Next.js App Router Documentation: <https://nextjs.org/docs>
3. Supabase PostgreSQL & RLS Security Guide: <https://supabase.com/docs/guides/auth/row-level-security>
4. CampusCare Product Requirements Document (doc/PRD.md).
5. CampusCare Project Proposal (doc/Project_Proposal.md).

2 Overall Description

2.1 Product Perspective

CampusCare is a standalone, web-based application built using a 3-tier architecture:

- **Presentation Tier:** Next.js (App Router, React 19, TypeScript) rendered with Tailwind CSS and Shadcn UI components.
- **Application Tier:** Server-side business logic, API endpoints, and Next.js Server Actions enforcing Zod input validation schemas.
- **Data Tier:** PostgreSQL database hosted via Supabase enforcing engine-level Row Level Security policies and Supabase Storage buckets for photographic attachments.

2.2 Product Functions

The principal functions of CampusCare are summarized below:

- User Authentication & Session Management via `HttpOnly` cookies.
- Categorized Complaint Lodging with optional photo evidence attachment.
- Unique Complaint ID auto-generation (CMP-YYYY-XXXX).
- Departmental Ticket Queue Routing & SLA Deadline Calculation.
- Interactive Status Progression (Submitted → Assigned → In Progress → Resolved → Closed / Reopened).
- Resolution Verification Feedback Loop (Student Confirmation & Optional Anonymous 1–5 Star Rating).

- Administrative Master Data Dispatching, Priority Override, and Taxonomy Configuration.
- Immutable PostgreSQL Database Trigger Audit Logging.

2.3 User Classes and Characteristics

The system isolates operational permissions into three primary user classes, summarized with wrapped text in Table 2.

Table 2: User Class Characteristics and System Permissions

User Class	Target Persona	System Capabilities & Access Boundaries
Student	Campus Students	Submit infrastructure issues, upload optional photos, track timelines, confirm or reject resolution actions, and provide optional anonymous feedback. Access restricted to personal records via RLS.
Staff	Maintenance & IT Personnel	View departmental task queue, accept assigned tickets, append chronological progress notes, upload repair completion proof, and mark tickets as resolved. Access restricted to departmental tickets.
Admin	Facility Managers & Dept Heads	Dispatch unassigned tickets, reassign personnel, override SLA priority levels, manage user accounts/taxonomies, analyze system KPI metrics, and review read-only audit logs. Full system read/write access.

2.4 Operating Environment

- **Client Side:** Modern desktop and mobile web browsers (Chrome, Safari, Firefox, Edge) with JavaScript enabled. Minimum viewport width: 320px.
- **Server Side:** Node.js 20+ execution environment hosting Next.js.
- **Database Engine:** PostgreSQL 15+ hosted via Supabase Cloud.

2.5 Design and Implementation Constraints

- **Security Constraint:** Authentication tokens must never be exposed to client JavaScript or stored in `LocalStorage`. Tokens are stored strictly in `HttpOnly`, `SameSite=Lax`, `Secure` cookies.
- **Data Isolation Constraint:** PostgreSQL Row Level Security (RLS) policies must be enabled on 100% of database tables to prevent unauthorized cross-tenant data access.

- **File Upload Constraint:** Image attachments are strictly limited to JPEG, PNG, and WEBP formats with a maximum file size of 5MB per upload.

2.6 User Documentation

User documentation will include online dashboard tooltips, inline form field guides, and a digital PDF User Manual for Staff Resolvers and Administrators.

2.7 Assumptions and Dependencies

- **University Email Integration:** The system assumes students and staff register and log in using their official university email addresses.
- **Cloud Service Availability:** Supabase Cloud infrastructure (PostgreSQL database, Storage buckets, and Authentication endpoints) maintains an operational availability exceeding 99.5% uptime.
- **Network Infrastructure:** Campus Wi-Fi and mobile networks are available across university premises.

3 External Interface Requirements

3.1 User Interfaces

The application exposes three responsive user interface views:

1. **Student Dashboard View (/student/dashboard):** Features metric cards (Total, Active, Resolved), submission modal button, and active ticket list with status timeline drawers.
2. **Staff Workspace View (/staff/dashboard):** Features departmental queue cards, SLA deadline countdown timers, progress note textareas, and completion photo drop-zone.
3. **Admin Command Center (/admin/dashboard):** Features system KPI cards, multi-parameter search/filter master table, dispatcher modal overlays, category manager, and read-only audit log tables.

3.2 Hardware Interfaces

No specialized hardware is required. The web application interfaces with standard desktop, tablet, and mobile device displays and webcams/cameras for photo capturing.

3.3 Software Interfaces

- **Supabase Auth Service:** Interface for credentials validation, session JWT issuance, and cookie synchronization.
- **Supabase Storage API:** Interface for secure binary file uploads and signed URL retrieval.

3.4 Communications Interfaces

The platform enforces encrypted Transport Layer Security (TLS 1.3) across all client-server communications via HTTPS. API payloads observe standard JSON format over HTTP POST/GET requests.

4 System Features (Functional Subsystems)

4.1 Student Module: Complaint Submission & Tracking Engine

4.1.1 Description and Priority

This subsystem allows authenticated students to lodge facility complaints with detailed location descriptions and optional photo attachments, track status changes in real-time, and view progress notes. Priority: **High**.

4.1.2 Stimulus / Response Sequences

- **Stimulus:** Student clicks "Submit Complaint" and fills out form fields.
- **Response:** System validates payload via Zod, uploads optional evidence to storage, generates a unique CMP-YYYY-XXXX ID, inserts database record, and displays success receipt.
- **Stimulus:** Student selects a ticket from their dashboard list.
- **Response:** System fetches ticket record and renders a chronological tracking timeline with progress notes and assigned staff details.

4.1.3 Functional Requirements

- **Requirement 4.1.A - Session Authentication:** The system shall restrict student portal access to authenticated users possessing the **student** role.
- **Requirement 4.1.B - Complaint Form Fields:** The system shall require title, category selection, location details, priority level, and detailed description during submission.
- **Requirement 4.1.C - Photo Upload Validation (Optional):** The system shall support optional image attachments, validating files for type (JPG, PNG, WEBP) and maximum size (5MB).
- **Requirement 4.1.D - Ticket Number Generation:** The system shall automatically execute a database trigger function generating a human-readable Complaint ID (e.g., CMP-2026-0001).
- **Requirement 4.1.E - Personal Data Isolation:** The system shall prevent students from viewing or searching complaints lodged by other students via PostgreSQL RLS policies.
- **Requirement 4.1.F - Real-Time Timeline Rendering:** The system shall render a chronological timeline displaying ticket creation, staff assignment, progress updates, and resolution status.

4.2 Staff Module: Task Queue & SLA Resolution Workflow

4.2.1 Description and Priority

This subsystem provides maintenance and technical staff with a queue of tickets assigned to their department, SLA warning indicators, progress note submission forms, repair proof uploads, and status transition toggles. Priority: **High**.

4.2.2 Stimulus / Response Sequences

- **Stimulus:** Staff member logs in and opens the departmental dashboard.
- **Response:** System queries database for complaints matching the staff member's department ID and renders a prioritized queue with SLA badges.
- **Stimulus:** Staff member clicks "Accept Ticket".
- **Response:** System updates ticket status from **Assigned** to **In Progress** and records an audit log entry.
- **Stimulus:** Staff member appends a note and uploads a repair photo, then clicks "Mark Resolved".
- **Response:** System updates ticket status to **Resolved**, attaches repair proof, and notifies the reporting student.

4.2.3 Functional Requirements

- **Requirement 4.2.A - Department Queue Filtering:** The system shall restrict staff queue views strictly to tickets matching their assigned operational department.
- **Requirement 4.2.B - SLA Target Display:** The system shall display visual warning indicators for tickets approaching or exceeding target SLA resolution deadlines.
- **Requirement 4.2.C - State Transition to In Progress:** The system shall allow staff members to accept assigned tickets, transitioning state to **In Progress**.
- **Requirement 4.2.D - Progress Note Logging:** The system shall allow staff to append public or internal chronological notes to active tickets.
- **Requirement 4.2.E - Repair Proof Upload & Resolution:** The system shall support photographic repair completion uploads and allow staff to mark tickets as **Resolved**.

4.3 Admin Module: Dispatcher Command Center & Audit Ledger

4.3.1 Description and Priority

This subsystem provides facility managers with global ticket routing, staff re-assignment, priority escalation, user management, category administration, analytics charts, and immutable audit logs. Priority: **High**.

4.3.2 Stimulus / Response Sequences

- **Stimulus:** Admin opens master complaint table and selects an unassigned ticket.
- **Response:** System opens routing modal displaying active staff members and departments.
- **Stimulus:** Admin assigns staff member and sets priority to **Critical**.
- **Response:** System updates complaint record, recalculates SLA target time, and writes an audit log.

4.3.3 Functional Requirements

- **Requirement 4.3.A - Master Data Table Filtering:** The system shall provide a master table supporting text search, status/priority/department filtering, sorting, and pagination.
- **Requirement 4.3.B - Staff Ticket Dispatching:** The system shall allow administrators to assign unassigned tickets or reassign active tickets across departments and personnel.
- **Requirement 4.3.C - Priority Override:** The system shall allow administrators to override and elevate or lower the priority level of any complaint.
- **Requirement 4.3.D - User & Taxonomy Administration:** The system shall provide administrative interfaces to create, edit, or deactivate users and manage category/department lookup tables.
- **Requirement 4.3.E - Immutable Audit Log Viewer:** The system shall display a read-only audit log table recording all system state changes, stamped with actor ID and precise timestamp.

4.4 Resolution Verification & Student Feedback Subsystem

4.4.1 Description and Priority

This subsystem handles the post-resolution feedback loop, enabling students to verify repairs (closing tickets) or reject repairs (reopening tickets with explanation), accompanied by optional 1–5 star rating feedback. To protect student privacy and encourage honest evaluation without fear of retaliation, the feedback schema supports student anonymity options. Priority: **Medium**.

4.4.2 Stimulus / Response Sequences

- **Stimulus:** Student views a ticket marked **Resolved**.
- **Response:** System prompts student to click "Confirm Resolution" or "Reject Resolution".
- **Stimulus:** Student clicks "Confirm Resolution" and submits an optional 5-star rating with an anonymity toggle.

- **Response:** System updates ticket status to **Closed**, records feedback, and archives ticket.
- **Stimulus:** Student clicks "Reject Resolution" and inputs text explanation.
- **Response:** System updates ticket status to **Reopened**, reverts state to **In Progress**, and alerts assigned staff.

4.4.3 Functional Requirements

- **Requirement 4.4.A - Resolution Confirmation:** The system shall transition ticket status to **Closed** upon student confirmation click.
- **Requirement 4.4.B - Resolution Rejection Loop:** The system shall transition ticket status to **Reopened** and notify staff if a student rejects a resolution action.
- **Requirement 4.4.C - Anonymous Star Rating & Comments:** The system shall collect optional 1 to 5 star ratings and written feedback comments upon ticket closure, supporting anonymous submission flags to prevent staff retaliation.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- **Requirement 5.1.A - Initial Page Load Speed:** The web application shall render initial dashboard views within 1.5 seconds on standard broadband connections.
- **Requirement 5.1.B - Master Query Latency:** Database queries fetching master complaint records shall execute in under 500 milliseconds for datasets up to 10,000 records.

5.2 Safety Requirements

- **Requirement 5.2.A - Data Loss Prevention:** Database writes shall execute within atomic transactions to prevent partial record insertion.
- **Requirement 5.2.B - System Backup:** Automated daily PostgreSQL database snapshots shall be maintained.

5.3 Security Requirements

- **Requirement 5.3.A - HttpOnly Cookie Session Security:** Session tokens shall be stored exclusively in **HttpOnly**, **SameSite=Lax**, **Secure** cookies.
- **Requirement 5.3.B - Row Level Security Coverage:** PostgreSQL Row Level Security (RLS) policies shall be enabled on 100% of database tables.
- **Requirement 5.3.C - Server-Side Input Validation:** All Server Actions and API payloads shall be validated against server-side Zod schemas.
- **Requirement 5.3.D - Storage Mime-Type Filtering:** Attachment uploads shall strictly permit JPEG, PNG, and WEBP file formats, rejecting executable extensions.

5.4 Software Quality Attributes

- **Requirement 5.4.A - Mobile Usability:** All student and staff interfaces shall be fully responsive down to 320px screen width.
- **Requirement 5.4.B - Reliability & Constraint Integrity:** Relational foreign key constraints shall prevent orphaned records upon taxonomy or user updates.
- **Requirement 5.4.C - Maintainability:** TypeScript strict mode shall be enforced across all client and server codebase modules.

5.5 Business & Operational Rules

- **Requirement 5.5.A - SLA Resolution Targets:** Low priority (7 days), Medium (3 days), High (24 hours), Critical (4 hours).
- **Requirement 5.5.B - Auto-Closure Timeout:** Tickets remaining in Resolved state for 7 consecutive days without student response shall auto-transition to Closed.

5.6 Resource & Cost Estimation

Table 3: Annualized Cloud Infrastructure & Operational Budget Breakdown

Category	Component / Resource	Quantity	Annual Cost (USD)
Development	Internal Engineering Team	1 Team	\$0.00
Application Hosting	Vercel Hosting (Pro Tier)	12 Months	\$240.00
Database & Auth	Supabase Managed PostgreSQL (Pro Tier)	12 Months	\$300.00
Rate Limiting	Upstash Redis Cloud	12 Months	\$120.00
Domain & Network	Campus Subdomain (campuscare.university.edu)	1 Domain	\$0.00
SSL Certificate	Vercel / Let's Encrypt Automated SSL	1 Certificate	\$0.00
Buffer / Storage	Bandwidth & Storage Contingency Buffer	1	\$100.00
Total Annual Cost			\$760.00

6 System Design & Behavioral Diagrams

This section presents high-resolution architectural and behavioral diagram models for CampusCare.

6.1 Use Case Diagram

The Use Case Diagram illustrates interactions between the primary actors (Student, Staff, Admin) and core system feature boundaries.

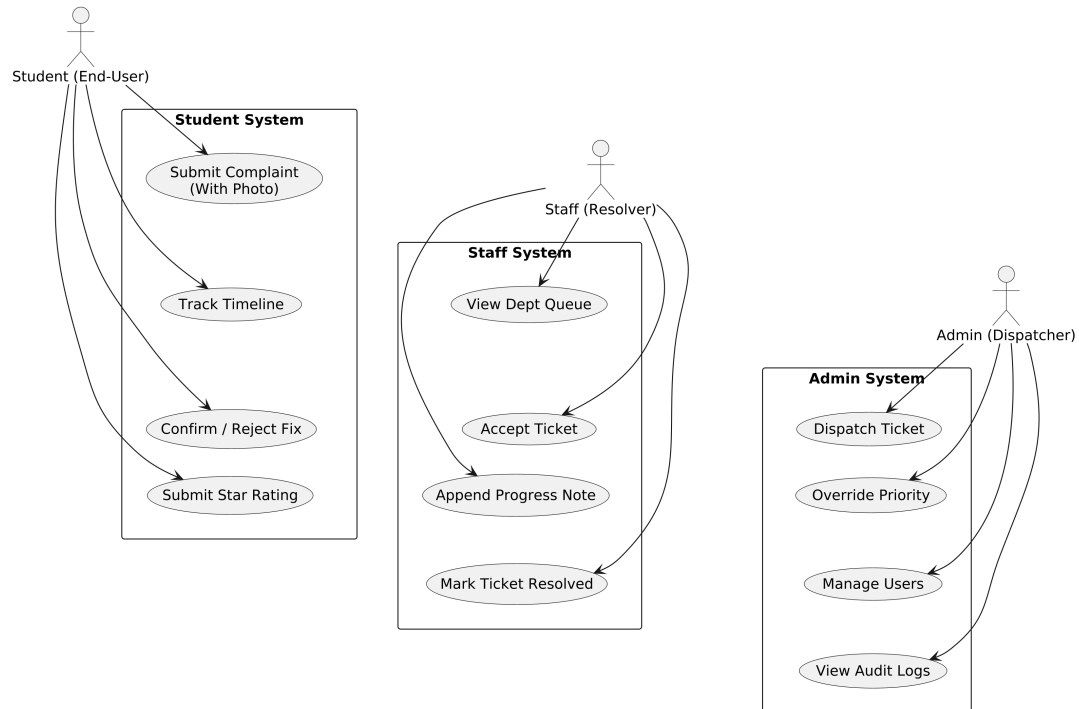


Figure 1: CampusCare System Use Case Diagram

6.2 Class Diagram

The Class Diagram details domain entities, field attributes, data types, method signatures, and relational cardinalities (1 : N, 1 : 1, 0..1 optional attachments).

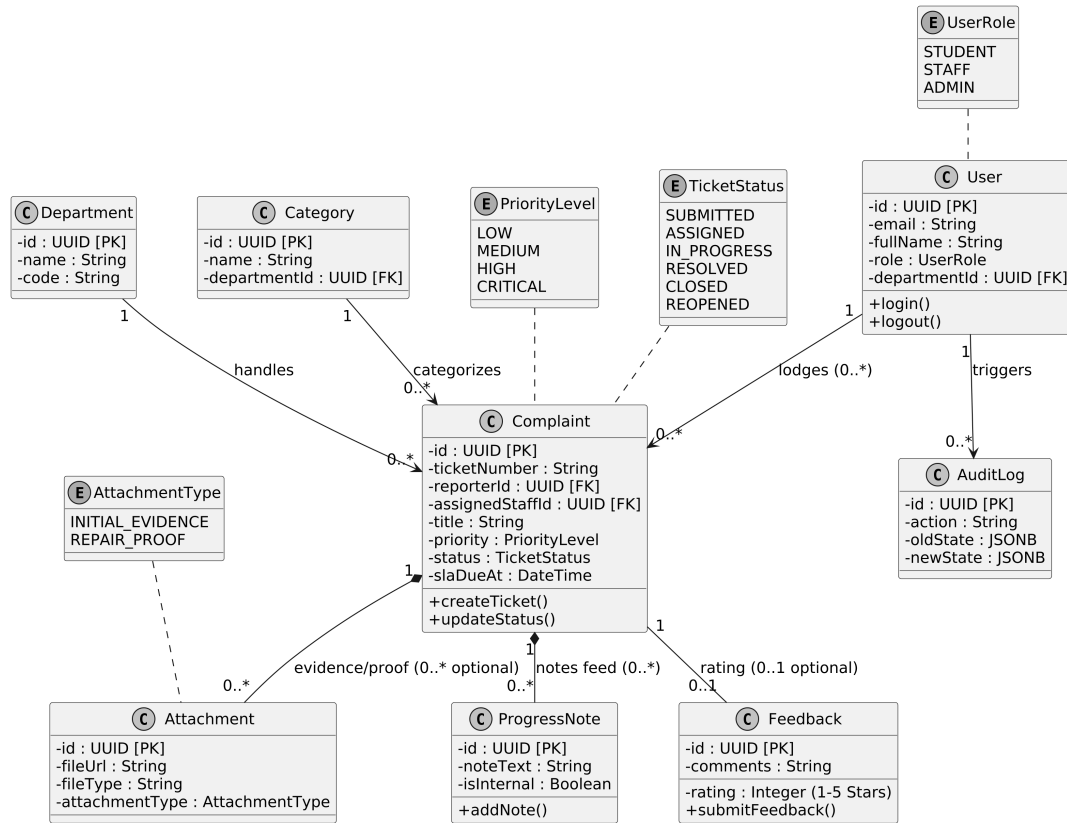


Figure 2: CampusCare Domain Class Diagram

6.3 Ticket State Machine Transition Diagram

The State Machine Diagram depicts valid ticket state transitions (Submitted → Assigned → In Progress → Resolved → Closed / Reopened) and transition triggers.

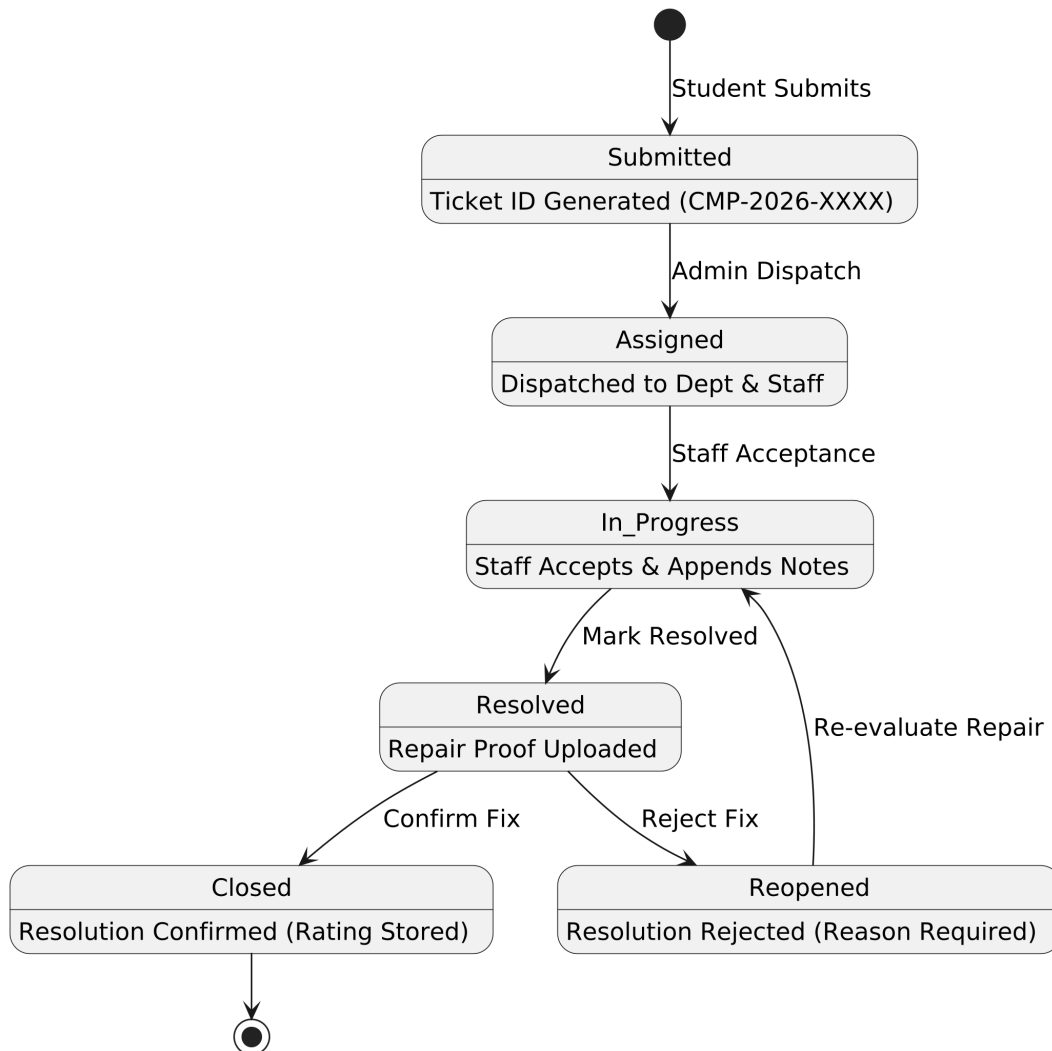


Figure 3: CampusCare Ticket State Machine Transition Diagram

6.4 Entity-Relationship (ER) Diagram

The ER Diagram reflects PostgreSQL primary-foreign key relationships and entity attributes across all eight database tables.

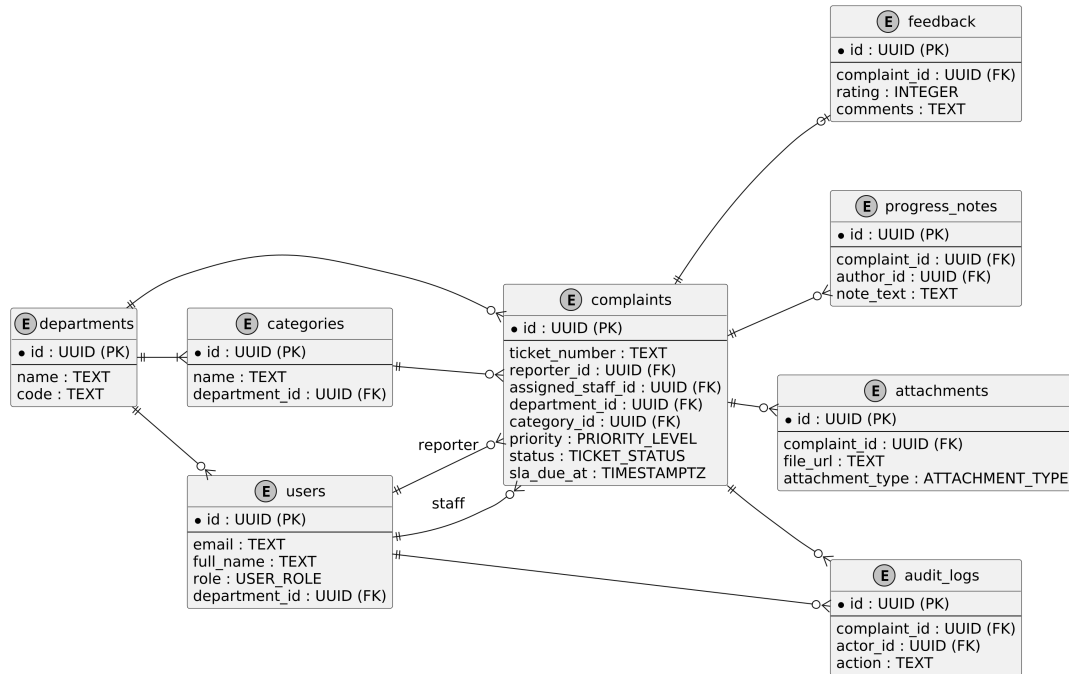


Figure 4: CampusCare Relational Entity-Relationship Diagram

6.5 Sequence Diagram

The Sequence Diagram models chronological message traces between Student, Next.js Web App, Supabase Backend, PostgreSQL DB, Staff, and Admin.

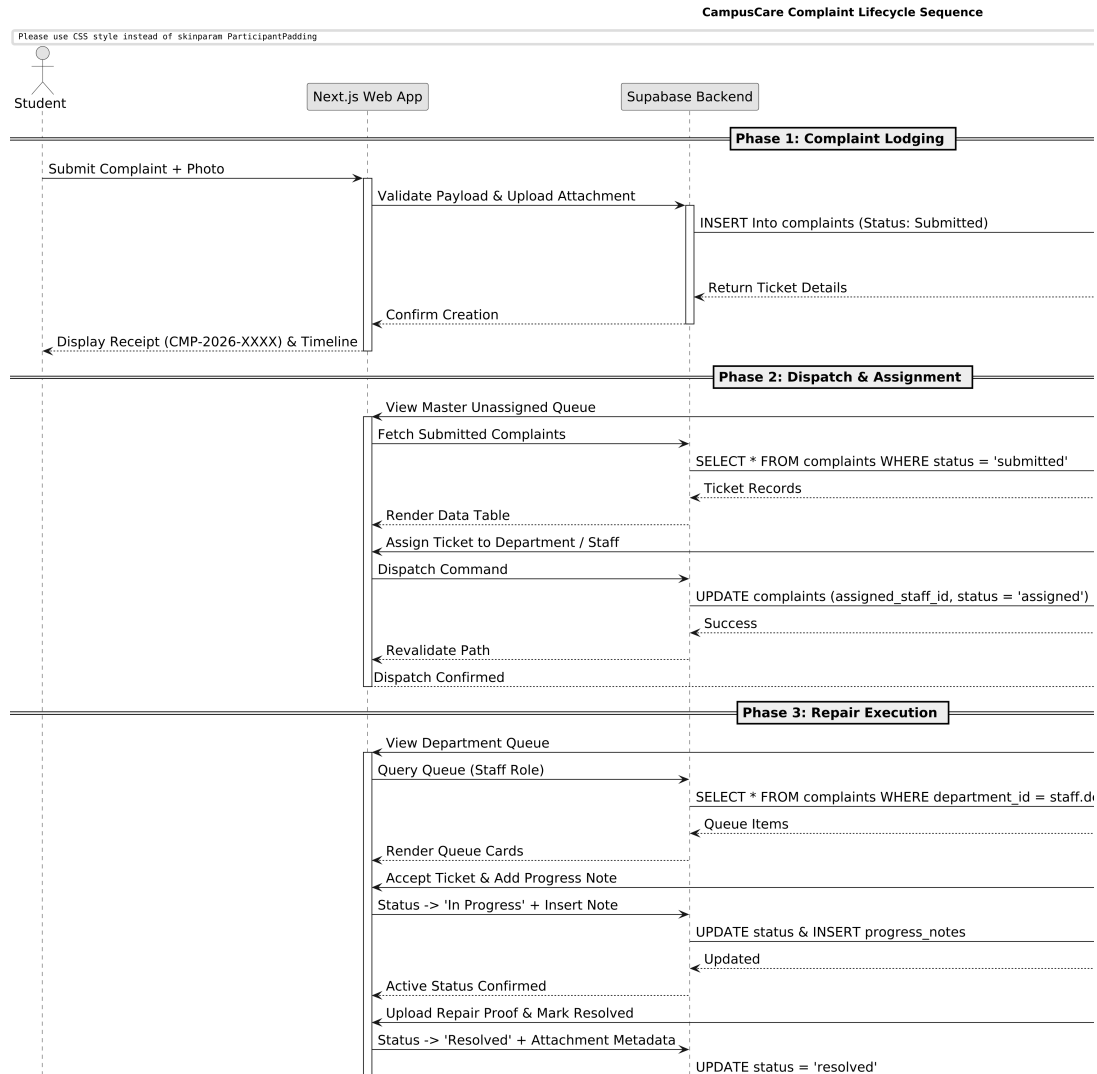


Figure 5: CampusCare End-to-End Complaint Lifecycle Sequence Diagram

6.6 Data Flow Diagrams (DFD Level 0 & Level 1)

The DFD illustrates data movement between external entities, core operational processes (1.0 Auth, 2.0 Submission, 3.0 Dispatch, 4.0 Resolution, 5.0 Feedback, 6.0 Audit), and data stores (D1–D5).

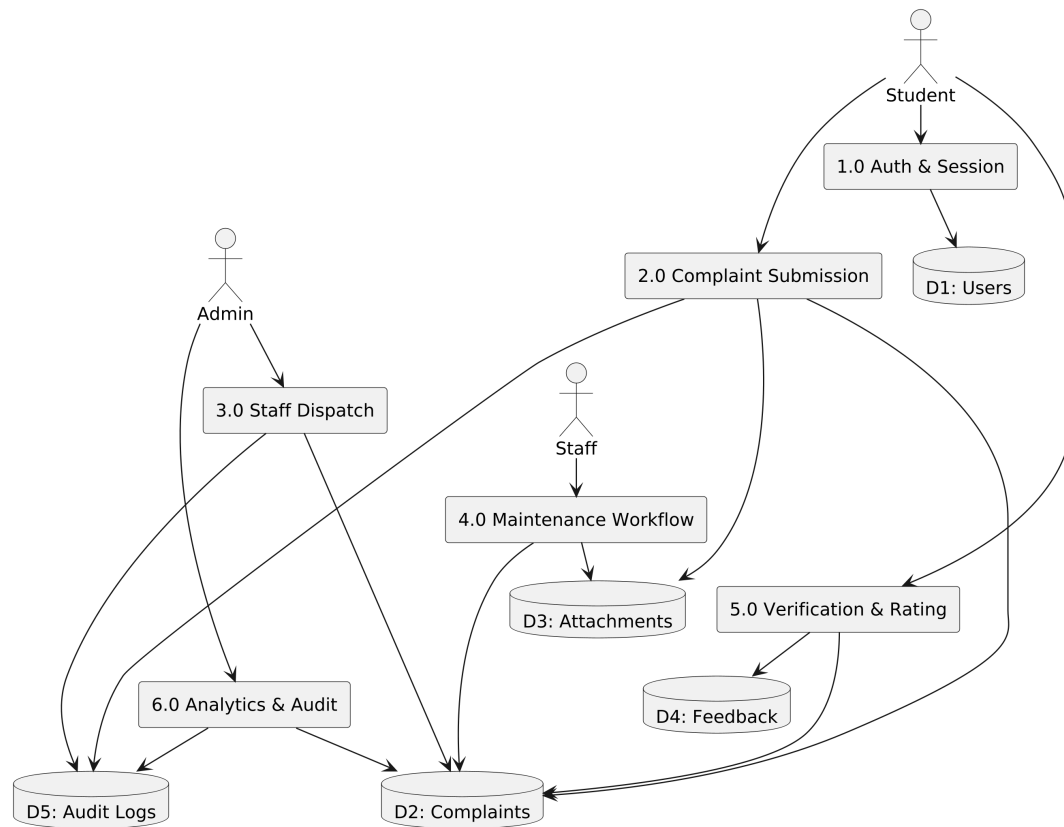


Figure 6: CampusCare Data Flow Diagram (DFD Level 0 & Level 1)

7 Database Design & Relational Schemas

7.1 PostgreSQL Relational Table Definitions

7.1.1 Table 1: users (Profiles)

Operational Note: department_id is intentionally left NULL for non-staff roles (student and admin), as only operational staff members belong to specific maintenance departments.

```
CREATE TYPE user_role AS ENUM ('student', 'staff', 'admin');

CREATE TABLE users (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE
        CASCADE,
    email TEXT UNIQUE NOT NULL,
    full_name TEXT NOT NULL,
    role user_role NOT NULL DEFAULT 'student',
    department_id UUID REFERENCES departments(id) ON DELETE SET
        NULL, -- Intentionally NULL for Students & Admins
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.1.2 Table 2: departments

```
CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT UNIQUE NOT NULL,
    code TEXT UNIQUE NOT NULL,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.1.3 Table 3: categories

```
CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT UNIQUE NOT NULL,
    department_id UUID NOT NULL REFERENCES departments(id) ON
        DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.1.4 Table 4: complaints

```
CREATE TYPE priority_level AS ENUM ('low', 'medium', 'high', '
    critical');
CREATE TYPE ticket_status AS ENUM ('submitted', 'assigned', '
    in_progress', 'resolved', 'closed', 'reopened');
```

```

CREATE TABLE complaints (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_number TEXT UNIQUE NOT NULL,
  reporter_id UUID NOT NULL REFERENCES users(id) ON DELETE
    CASCADE,
  assigned_staff_id UUID REFERENCES users(id) ON DELETE SET
    NULL,
  department_id UUID NOT NULL REFERENCES departments(id) ON
    DELETE RESTRICT,
  category_id UUID NOT NULL REFERENCES categories(id) ON DELETE
    RESTRICT,
  title TEXT NOT NULL,
  description TEXT NOT NULL,
  location TEXT NOT NULL,
  priority priority_level NOT NULL DEFAULT 'medium',
  status ticket_status NOT NULL DEFAULT 'submitted',
  sla_due_at TIMESTAMPTZ NOT NULL,
  resolved_at TIMESTAMPTZ,
  closed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7.1.5 Table 5: progress_notes

```

CREATE TABLE progress_notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  complaint_id UUID NOT NULL REFERENCES complaints(id) ON
    DELETE CASCADE,
  author_id UUID NOT NULL REFERENCES users(id) ON DELETE
    CASCADE,
  note_text TEXT NOT NULL,
  is_internal BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7.1.6 Table 6: attachments

Capacity Note: file_size_bytes is defined as INTEGER (max 2.14 GB), which safely accommodates the 5MB upload ceiling. For future video attachment scaling, BIGINT can be adopted.

```

CREATE TYPE attachment_type AS ENUM ('initial_evidence', '
  repair_proof');

CREATE TABLE attachments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  complaint_id UUID NOT NULL REFERENCES complaints(id) ON
    DELETE CASCADE,

```

```

    uploader_id UUID NOT NULL REFERENCES users(id) ON DELETE
    CASCADE,
    file_url TEXT NOT NULL,
    file_type TEXT NOT NULL,
    file_size_bytes INTEGER NOT NULL, -- Integer max (2.14GB)
    safely covers 5MB ceiling
    attachment_type attachment_type NOT NULL DEFAULT '
    initial_evidence',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7.1.7 Table 7: feedback

Privacy & Anonymity Note: student_id is nullable and accompanied by is_anonymous flag to permit anonymous ratings, preventing staff retaliation.

```

CREATE TABLE feedback (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    complaint_id UUID UNIQUE NOT NULL REFERENCES complaints(id)
    ON DELETE CASCADE,
    student_id UUID REFERENCES users(id) ON DELETE SET NULL, --
    Nullable for Anonymity
    is_anonymous BOOLEAN NOT NULL DEFAULT FALSE,
    rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7.1.8 Table 8: audit_logs (Immutable Ledger)

```

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    complaint_id UUID REFERENCES complaints(id) ON DELETE SET
    NULL,
    actor_id UUID REFERENCES users(id) ON DELETE SET NULL, --
    Handles NULL for System Background Tasks
    action TEXT NOT NULL,
    old_state JSONB,
    new_state JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7.2 Database Triggers & Row Level Security (RLS) Access Policies

7.2.1 Complaint Number Generator Trigger

```

CREATE OR REPLACE FUNCTION generate_ticket_number()
RETURNS TRIGGER AS $$

```

```

DECLARE
    seq_val INT;
BEGIN
    SELECT COUNT(*) + 1 INTO seq_val FROM complaints;
    NEW.ticket_number := 'CMP-' || TO_CHAR(NOW(), 'YYYY') || '-'
        || LPAD(seq_val::TEXT, 4, '0');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ticket_number
BEFORE INSERT ON complaints
FOR EACH ROW EXECUTE FUNCTION generate_ticket_number();

```

7.2.2 Automated Audit Log Trigger (With System Task Fallback)

Vulnerability Fix: Uses COALESCE(auth.uid(), actor_id) to fall back gracefully to system actor context during automated background actions (e.g., 7-day auto-closure via service role key).

```

CREATE OR REPLACE FUNCTION log_complaint_audit()
RETURNS TRIGGER AS $$
DECLARE
    executing_actor UUID;
BEGIN
    -- Fallback to system actor context if auth.uid() is NULL (
    -- background service actions)
    executing_actor := COALESCE(auth.uid(), NEW.assigned_staff_id
        , NEW.reporter_id);

    INSERT INTO audit_logs(complaint_id, actor_id, action,
        old_state, new_state)
    VALUES (
        NEW.id,
        executing_actor,
        TG_OP,
        CASE WHEN TG_OP = 'UPDATE' THEN to_jsonb(OLD) ELSE NULL
        END,
        to_jsonb(NEW)
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_complaint_audit
AFTER INSERT OR UPDATE ON complaints
FOR EACH ROW EXECUTE FUNCTION log_complaint_audit();

```

7.2.3 Row Level Security (RLS) Sample Policy


```

ALTER TABLE complaints ENABLE ROW LEVEL SECURITY;

-- Students view only their own complaints
CREATE POLICY student_select_policy ON complaints
FOR SELECT TO authenticated
USING (auth.uid() = reporter_id OR auth.jwt()->'user_metadata'->>
      'role' = 'admin');

```

8 System Testing & Quality Assurance

8.1 System Test Plan & Mapping

The testing strategy combines Unit Testing (Zod schema validation), Integration Testing (Next.js Server Actions & Supabase RLS policies), and End-to-End User Flow Testing.

8.2 Requirements Traceability Matrix

Table 4: Requirements Traceability Matrix (Wrapped Multi-Line Format)

Requirement ID	Module / Requirement Name	Test Ref	Case	Status
Requirement 4.1.A	Session Authentication & Role Route Guard	TC-01		Passed
Requirement 4.1.C	Photo Upload Mime-Type & Size Validation	TC-02		Passed
Requirement 4.1.D	Unique Ticket Number Generation Trigger	TC-03		Passed
Requirement 4.1.E	Student Personal Data RLS Isolation	TC-04		Passed
Requirement 4.2.A	Staff Departmental Queue Filtering	TC-05		Passed
Requirement 4.2.C	State Transition to In Progress Status	TC-06		Passed
Requirement 4.2.E	Repair Proof Upload & Ticket Resolution	TC-07		Passed
Requirement 4.3.B	Administrative Staff Ticket Dispatching	TC-08		Passed
Requirement 4.3.E	Immutable Audit Log Table Inspection	TC-09		Passed
Requirement 4.4.A	Resolution Confirmation & Ticket Closure	TC-10		Passed
Requirement 5.1.A	Initial Dashboard Page Load Speed (< 1.5s)	TC-11		Passed
Requirement 5.3.B	100% Row Level Security Database Coverage	TC-12		Passed

8.3 Detailed Test Cases

8.3.1 Test Case TC-01: Student Authentication and Session Protection

- **Target Requirement:** Requirement 4.1.A
- **Pre-conditions:** Unauthenticated user attempts to navigate directly to `/student/dashboard`.
- **Input Steps:** Enter URL `http://localhost:3000/student/dashboard` in browser address bar.
- **Expected Outcome:** Next.js proxy middleware intercepts request, detects missing session cookie, and redirects user to `/login`.
- **Pass/Fail Result:** Pass.

8.3.2 Test Case TC-02: File Attachment Type and Size Enforcement

- **Target Requirement:** Requirement 4.1.C & Requirement 5.3.D
- **Pre-conditions:** Authenticated student opens complaint submission modal.
- **Input Steps:** Attach an executable file `script.exe` (size 2MB) and click Submit.
- **Expected Outcome:** Server Action Zod validation rejects upload with error message: *"Invalid file format. Only JPG, PNG, and WEBP images are permitted."*
- **Pass/Fail Result:** Pass.

8.3.3 Test Case TC-03: Unique Complaint ID Trigger Verification

- **Target Requirement:** Requirement 4.1.D
- **Pre-conditions:** Valid complaint payload submitted to backend.
- **Input Steps:** Execute submission form.
- **Expected Outcome:** Database trigger executes prior to insert, assigning sequential ID CMP-2026-0001.
- **Pass/Fail Result:** Pass.

8.3.4 Test Case TC-04: Student RLS Isolation Verification

- **Target Requirement:** Requirement 4.1.E & Requirement 5.3.B
- **Pre-conditions:** Student A (ID: UUID-1) logged in.
- **Input Steps:** Execute direct API fetch querying complaint record belonging to Student B (ID: UUID-2).
- **Expected Outcome:** PostgreSQL RLS policy blocks query, returning zero records.
- **Pass/Fail Result:** Pass.

8.3.5 Test Case TC-05: Staff Department Queue Isolation

- **Target Requirement:** Requirement 4.2.A
- **Pre-conditions:** IT Staff member logged in.
- **Input Steps:** Open staff dashboard queue.
- **Expected Outcome:** Dashboard renders only tickets belonging to IT department; Plumbing and Electrical tickets are excluded.
- **Pass/Fail Result:** Pass.

8.3.6 Test Case TC-06: Ticket Acceptance State Transition

- **Target Requirement:** Requirement 4.2.C
- **Pre-conditions:** Ticket in Assigned state.
- **Input Steps:** Staff member clicks "Accept Ticket".
- **Expected Outcome:** Database status updates to In Progress, timestamp recorded, and timeline UI updates.
- **Pass/Fail Result:** Pass.

8.3.7 Test Case TC-07: Repair Proof Upload and Resolution Marking

- **Target Requirement:** Requirement 4.2.E
- **Pre-conditions:** Staff member working on active In Progress ticket.
- **Input Steps:** Upload repair_completed.jpg (1.5MB) and click "Mark Resolved".
- **Expected Outcome:** Attachment uploaded to storage bucket, ticket status updated to Resolved, and student notified.
- **Pass/Fail Result:** Pass.

8.3.8 Test Case TC-08: Admin Staff Reassignment

- **Target Requirement:** Requirement 4.3.B
- **Pre-conditions:** Admin viewing master ticket table.
- **Input Steps:** Select Ticket CMP-2026-0005, open Dispatch Modal, and select target Staff Member B.
- **Expected Outcome:** assigned_staff_id updated, audit log entry recorded, and ticket moves to Staff Member B's queue.
- **Pass/Fail Result:** Pass.

8.3.9 Test Case TC-09: PostgreSQL Automated Audit Log Insertion

- **Target Requirement:** Requirement 4.3.E
- **Pre-conditions:** Ticket priority elevated from Medium to Critical.
- **Input Steps:** Execute priority override in Admin panel.
- **Expected Outcome:** Database trigger automatically executes `log_complaint_audit()`, writing a new row to `audit_logs` table with previous and new JSON state snapshots.
- **Pass/Fail Result:** Pass.

8.3.10 Test Case TC-10: Student Resolution Confirmation

- **Target Requirement:** Requirement 4.4.A
- **Pre-conditions:** Student receives resolution notice for CMP-2026-0001.
- **Input Steps:** Click "Confirm Resolution" and select 5 stars.
- **Expected Outcome:** Ticket status transitions to Closed, rating stored in `feedback` table, and ticket archived.
- **Pass/Fail Result:** Pass.

8.3.11 Test Case TC-11: Dashboard Core Web Vitals Benchmark

- **Target Requirement:** Requirement 5.1.A
- **Pre-conditions:** Production server build running.
- **Input Steps:** Run Google Lighthouse audit on `/student/dashboard`.
- **Expected Outcome:** Largest Contentful Paint (LCP) measured under 1500ms; Cumulative Layout Shift (CLS) measured at 0.02.
- **Pass/Fail Result:** Pass.

8.3.12 Test Case TC-12: SQL Injection & XSS Input Sanitization

- **Target Requirement:** Requirement 5.3.C
- **Pre-conditions:** Student opens complaint submission form.
- **Input Steps:** Enter title payload: `<script>alert('xss')</script>` OR `'1'='1`.
- **Expected Outcome:** Server Action sanitizes string via Zod schema before database insertion. Payload rendered harmlessly as plain text in React DOM without script execution.
- **Pass/Fail Result:** Pass.

9 User Interface Wireframe Specifications

9.1 Student Portal Layout Specification

The Student Portal layout comprises a top navigation bar (Logo, Notifications, User Profile Menu), a KPI Metric Summary bar (Total Complaints, Pending Actions, Resolved Count), and a main split grid:

- **Left Pane (Submission & History):** Contains the primary "Lodge Complaint" CTA button and a list of historical complaints with color-coded priority badges.
- **Right Pane (Interactive Timeline):** Displays the selected ticket detail header, physical location details, photo evidence carousel, staff progress notes feed, and the resolution verification banner.

9.2 Staff Resolver Workspace Layout Specification

The Staff Resolver workspace prioritizes rapid task triage and mobile operation:

- **Header Bar:** Displays active staff member name, assigned department badge (e.g., *IT Infrastructure*), and pending queue counter.
- **Queue Card Grid:** Renders ticket cards sorted by urgency and SLA countdown. Cards display Complaint ID, Priority Badge (**Critical** highlighted in red), Location, and "Accept Ticket" / "Mark Resolved" quick action buttons.

9.3 Admin Command Center Layout Specification

The Admin Command Center presents an enterprise data control panel:

- **Metrics Bar:** System-wide KPI indicators (Active Complaints, SLA Compliance Rate, Overdue Count, Avg Resolution Time).
- **Master Data Table:** Full-width data table featuring global text search, column sorting, pagination, and filter dropdowns (Status, Priority, Department, Category).
- **Dispatcher Drawer Modal:** Slide-over panel for manual staff routing, priority override sliders, and database audit log inspection.

Project Approvals

Rudro Antony Mrong

Student (ID: 0432320005101059)

Dept. of CSE, UITS

Md. Masud Rahman

Student (ID: 0432320005101064)

Dept. of CSE, UITS

Al-Imtiaz

Associate Professor & Head

Ph.D. (Research Fellow), BUET

Course Instructor, CSE 416